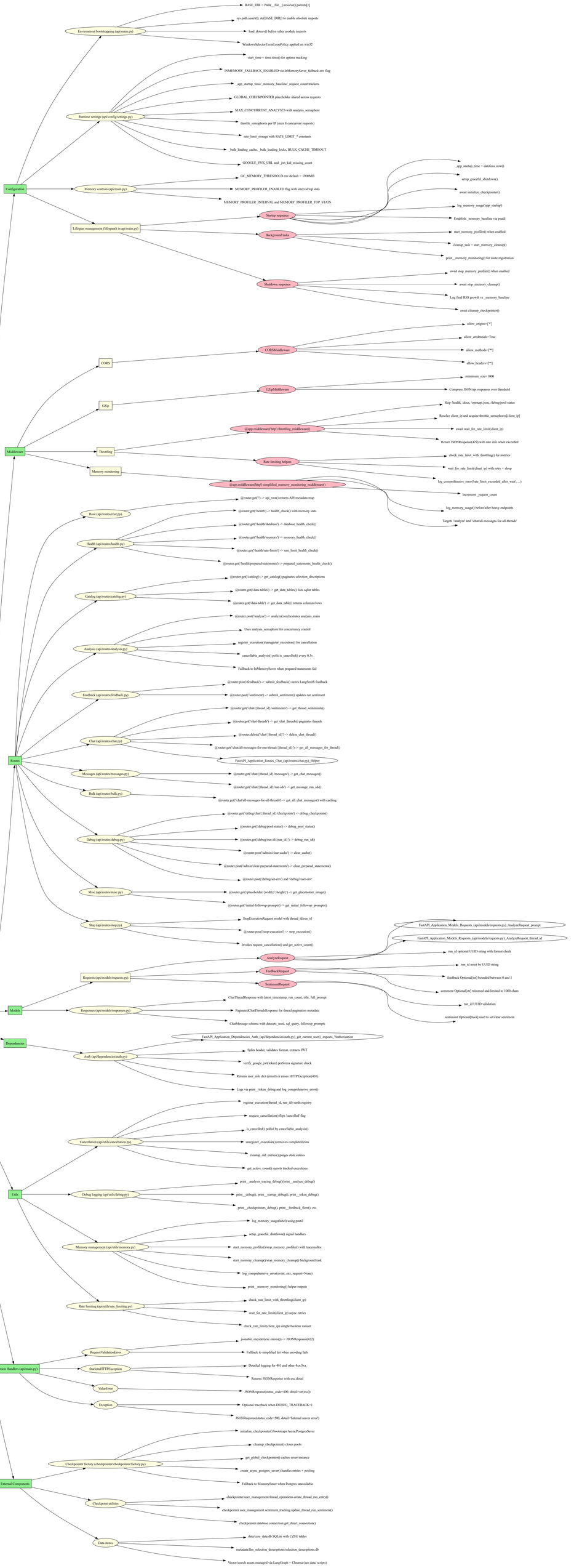
FastAPI Application
Configuration
Environment bootstrapping (api/main.py)
BASE_DIR = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(BASE_DIR)) to enable absolute imports
load_dotenv() before other module imports
WindowsSelectorEventLoopPolicy applied on win32
Runtime settings (api/config/settings.py)
start_time = time.time() for uptime tracking
INMEMORY_FALLBACK_ENABLED via InMemorySaver_fallback env flag
_app_startup_time/ _memory_baseline/ _request_count trackers
GLOBAL_CHECKPOINTER placeholder shared across requests
MAX_CONCURRENT_ANALYSES with analysis_semaphore
throttle_semaphores per IP (max 8 concurrent requests)
rate_limit_storage with RATE_LIMIT_* constants
_bulk_loading_cache, _bulk_loading_locks, BULK_CACHE_TIMEOUT
GOOGLE_JWK_URL and _jwt_kid_missing_count
Memory controls (api/main.py)
GC_MEMORY_THRESHOLD env default = 1900MB
MEMORY_PROFILER_ENABLED flag with interval/top stats
MEMORY_PROFILER_INTERVAL and MEMORY_PROFILER_TOP_STATS
Lifespan management (lifespan() in api/main.py)
Startup sequence
_app_startup_time = datetime.now()
setup_graceful_shutdown()
await initialize_checkpointer()
log_memory_usage('app_startup')
Establish _memory_baseline via psutil
Background tasks
start_memory_profiler() when enabled
cleanup_task = start_memory_cleanup()
print__memory_monitoring() for route registration
Shutdown sequence
await stop_memory_profiler() when enabled
await stop_memory_cleanup()
Log final RSS growth vs _memory_baseline
await cleanup_checkpointer()
Middleware
CORS
CORSMiddleware
allow_origins=["*"]
allow_credentials=True
allow_methods=["*"]
allow_headers=["*"]
GZip
GZipMiddleware
minimum_size=1000
Compress JSONrags responses over threshold
Throttling
@app.middleware('http') throttling_middleware()
Skip /health, /docs, /openapi.json, /debug/pool-status
Resolve client_ip and acquire throttle_semaphore[client_ip]
await wait_for_rate_limit(client_ip)
Return JSONResponse(429) with rate info when exceeded
Rate limiting helpers
check_rate_limit_with_throttling() for metrics
wait_for_rate_limit(client_ip) with retry + sleep
log_comprehensive_error('rate_limit_exceeded_after_wait', ...)
Memory monitoring
@app.middleware('http') simplified_memory_monitoring_middleware()
Increment _request_count
log_memory_usage() before/after heavy endpoints
Targets '/analyze' and '/chat/all-messages-for-all-threads'
Routes
Root (api/routes/root.py)
@router.get("/") -> api_root() returns API metadata map
Health (api/routes/health.py)
@router.get("/health") -> health_check() with memory stats
@router.get('/health/database') -> database_health_check()
@router.get('/health/memory') -> memory_health_check()
@router.get('/health/rate-limits') -> rate_limit_health_check()
@router.get('/health/prepared-statements') -> prepared_statements_health_check()
Catalog (api/routes/catalog.py)
@router.get("/catalog") -> get_catalog() paginates selection_descriptions
@router.get('/data-tables') -> get_data_tables() lists sqlite tables
@router.get('/data-table/') -> get_data_table() returns columns/rows
Analysis (api/routes/analysis.py)
@router.post("/analyze") -> analyze() orchestrates analysis_main
Uses analysis_semaphore for concurrency control
register_execution()/unregister_execution() for cancellation
cancellable_analysis() polls is_cancelled() every 0.5s
Fallback to InMemorySaver when prepared statements fail
Feedback (api/routes/feedback.py)
@router.post("/feedback") -> submit_feedback() stores LangSmith feedback
@router.post("/sentiment") -> submit_sentiment() updates run sentiment
Chat (api/routes/chat.py)
@router.get('/chat/{thread_id}/sentiments') -> get_thread_sentiments()
@router.get("/chat-threads") -> get_chat_threads() paginates threads
@router.delete('/chat/{thread_id}') -> delete_chat_thread()
@router.get("/chat/all-messages-for-one-thread/{thread_id}") -> get_all_messages_for_thread()
FastAPI_Application_Routes_Chat_(api/routes/chat.py)_Helper
Messages (api/routes/messages.py)
@router.get("/chat/{thread_id}/messages") -> get_chat_messages()
Bulk (api/routes/bulk.py)
@router.get("/chat/{thread_id}/run-idle") -> get_message_run_idle()
@router.get("/chat/all-messages-for-all-threads") -> get_all_chat_messages() with caching
Debug (api/routes/debug.py)
@router.get('/debug/chat/{thread_id}/checkpoints') -> debug_checkpoints()
@router.get("/debug/pool-status") -> debug_pool_status()
@router.get('/debug/run-id/{run_id}') -> debug_run_id()
@router.post("/admin/clear-cache") -> clear_cache()
@router.post("/admin/clear-prepared-statements") -> clear_prepared_statements()
@router.post("/debug/set-env") and '/debug/reset-env'
Misc (api/routes/misc.py)
@router.get("/placeholder/{width}/{height}") -> get_placeholder_image()
@router.get("/initial-followup-prompts") -> get_initial_followup_prompts()
Stop (api/routes/stop.py)
StopExecutionRequest model with thread_id/run_id
@router.post("/stop-execution") -> stop_execution()
Invokes request_cancellation() and get_active_count()
Models
Requests (api/models/requests.py)
AnalyzeRequest
FastAPI_Application_Models_Requests_(api/models/requests.py)_AnalyzeRequest_prompt
FastAPI_Application_Models_Requests_(api/models/requests.py)_AnalyzeRequest_thread_id
run_id optional UUID string with format check
FeedbackRequest
run_id must be UUID string
feedback Optional[int] bounded between 0 and 1
SentimentRequest
comment Optional[str] trimmed and limited to 1000 chars
run_id UUID validation
sentiment Optional[bool] used to set/clear sentiment
Responses (api/models/responses.py)
ChatThreadResponse with latest_timestamp, run_count, title, full_prompt
PaginatedChatThreadsResponse for thread pagination metadata
ChatMessage schema with datasets_used, sql_query, followup_prompts
get_current_user() expects 'Authorization: Bearer <token>'
thread_id: str (1-100 chars) stripped and validated
prompt: str (1-10000 chars) validated via field_validator
Helper: get_thread_messages_with_metadata() reused by other routes
Dependencies
FastAPI_Application_Dependencies_Auth_(api/dependencies/auth.py)_get_current_user()_expects_'Authorization
Auth (api/dependencies/auth.py)
Splits header, validates format, extracts JWT
verify_google_jwt(token) performs signature check
Returns user_info dict (email) or raises HTTPEXcepion(401)
Logs via print _token_debug and log_comprehensive_error()
Utils
Cancellation (api/utils/cancellation.py)
register_execution(thread_id, run_id) seeds registry
request_cancellation() flips 'cancelled' flag
is_cancelled() polled by cancellable_analysis()
unregister_execution() removes completed runs
cleanup_old_entries() purges stale entries
get_active_count() reports tracked executions
Debug logging (api/utils/debug.py)
print__analysis_tracing_debug()/print__analyze_debug()
print__debug(), print__startup_debug(), print__token_debug()
print__checkpointers_debug(), print__feedback_flow(), etc.
Memory management (api/utils/memory.py)
log_memory_usage(label) using psutil
setup_graceful_shutdown() signal handlers
start_memory_profiler()/stop_memory_profiler() with tracemalloc
start_memory_cleanup()/stop_memory_cleanup() background task
log_comprehensive_error(event, exc, request=None)
print__memory_monitoring() helper outputs
Rate limiting (api/utils/rate_limiting.py)
check_rate_limit_with_throttling(client_ip)
wait_for_rate_limit(client_ip) async retries
check_rate_limit(client_ip) simple boolean variant
Exception Handlers (api/main.py)
RequestValidationError
jsonable_encoder(exc.errors()) -> JSONResponse(422)
Fallback to simplified list when encoding fails
StarletteHTTPException
Detailed logging for 401 and other 4xx/5xx
Returns JSONResponse with exc.detail
ValueError
JSONResponse(status_code=400, detail=str(exc))
Exception
Optional traceback when DEBUG_TRACEBACK=1
JSONResponse(status_code=500, detail="Internal server error")
External Components
Checkpointer factory (checkpointer/checkpointer/factory.py)
initialize_checkpointer() bootstraps AsyncPostgresSaver
cleanup_checkpointer() closes pools
get_global_checkpointer() caches saver instance
create_async_postgres_saver() handles retries + pooling
Fallback to MemorySaver when Postgres unavailable
Checkpoint utilities
checkpointer.user_management.thread_operations.create_thread_run_entry()
checkpointer.user_management.sentiment_tracking.update_thread_run_sentiment()
checkpointer.database.connection.get_direct_connection()
Data stores
data/czsu_data.db SQLite with CZSU tables
metadata/llm_selection_descriptions/selection_descriptions.db
Vector/search assets managed via LangGraph + Chroma (see data/ scripts)